Today
* Process address space
* Stack variable allocations
* Pointers & addresses
* Pointer types
* NULL Pointer
* Array basics

Process address space:

two types of computer mem
1. volatile mem → RAM
2. Storage mem → SSD/HD

we will focus on 1. RAM!

512GB virtual memory
- when you run a file, the code gets run in the bottom portion of ram
- when a program gets loaded multiple times, these are Processes (Browser)
- Computers work in ~~VRAM~~ virtual memory
  - they map
  - each process thinks it is the only process on the computer running on the entire RAM using 512GB of Ram
- CPU & operating system handle mapping processes to map to VRAM Physical RAM.
- static vars get allocated before the rest of the process.
- the heap is something you call with malloc(). heap is manually controlled.
- Stack grows as vars are declared & used.

tiny portion
Code
Small portion of RAM not used.

RAM 16GB
really big space

512GB virtual memory
Stack
heap
Static Vars
Code
minimum size of Program

Local variables / Stack variables

Stack variable allocations

```
int f(){
  int x;
  int y;
  ⋮
}
int main(){
  int a = 100;
  int b = 200;
  ⋮
  f();
}
```

BP
SP
512 GB
a = 100
b = 200
x
y
Code

Stack grows down.
return address is stored.
- After f's y goes out of scope, the stack Pointer rolls back up to where it was initially.
- Compiler knows how big the stack needs to be implemented initially.
- Base Pointer points

Stack Pointer
- Points to the bottom of the stack
- handled by the CPU
- Each byte has an address

Pointer types

```
int i = 1234;
double d = 3.14;
int *pi = &i;
double *pd = &d
```

pi
1234
pd
3.14

pi thinks it only has this space. It will grab only the first part of 4 bytes of 8 byte double & interpret it as half of the double.

pi = pd; two distinct types, the Compiler will not let this Compile.

this Compiles → pi = (int*)pd; type casting operator

- Casting can be necessary, & it can also be unpredictable just learn when to use it

Special Pointer type that can take any type.

- you can not dereference a void *

```
void *pv = pi;
pd = pv;
int y = *(int*)pv;
```

You must cast the void Pointer to get the right type.

Pointers & Addresses

```
foo(){
  int x = 1;
  int *p;
  p = &x;
}
```

2nd 2000
x = 10 → x = 1;
p = 2000
1892
y = XXXX
1888
int *p;
p = &x;
int*

type is int*
- Pointer to int
- it can hold the address of another int variable.
- size of Pointers is 8bytes/64bit.

- unary operator address of operator
- grabs the address of the following var.

- addresses although nums are different than ints;
- You can take the address of any type, and it will have a unique type*

* - dereferencing operator
- follows the Pointer & grabs what it is referencing.

```
foo(){
  int x = 1;
  int *p;
  p = &x;
  y = *p;
  *p = 0;
  p = &y; // Pointer references y now
  ++*p;
  (*p)++;
}
```

int *p; → written like this because when you dereference it it will become an int.

Null Pointers

```
{
  int *p;
  p = 0;
  p = NULL;
  if(p){
    ⋮
    // true if p is not NULL
```

not a valid address & is never used. This is a null Pointer
NULL ≡ ∅

p → 2565 0

- Pointers just like integers
Can be used in a boolean expression

Understand this distinction

```
if(p) //T
if(*p) //F
if(q) //F
if(*q) //crash
```

```
int x = 0;
int *p = &x;
int *q = 0;
```

p → 0
q 0

null Pointer exception